

The Evolution of AI Agents and Modern AI Engineering

Technical Research Report — Week 1, Assignment 2

Hooria Zahoor | BS Artificial Intelligence, The University of Faisalabad | **Track 2: NLP & AI Agents**

ABSTRACT

Large Language Models (LLMs) have evolved from single-turn text generators into the reasoning engines of autonomous AI agents capable of planning, using external tools, remembering context, and collaborating in multi-agent systems. This report traces that evolution — from simple LLM applications to tool-augmented, memory-equipped, planning-driven agents — and examines the architectural patterns, real-world deployments, and emerging trends that define modern AI engineering. Specifically, it walks through seven interconnected topics: LLM applications, AI agents, tool calling, memory, planning, multi-agent systems, and the future direction of AI engineering, illustrating each with an architecture diagram and real-world examples drawn from current industry systems. Together, these developments mark a fundamental shift in AI engineering practice: from crafting individual prompts to designing complete, reliable, and scalable agentic systems.



1 LLM Applications

Large Language Models (LLMs) such as GPT-4, Claude, and Gemini are foundation models trained on vast text corpora to predict and generate coherent language. Their earliest and most direct applications were **single-shot text tasks**: answering questions, summarizing documents, translating languages, generating code snippets, and drafting content. These applications treat the LLM as a stateless function — a prompt goes in, a completion comes out, with no persistent memory or ability to act on the world.

Common categories of LLM applications:

- **Conversational assistants** — ChatGPT, Claude.ai, Gemini for Q&A and dialogue.
- **Content & code generation** — drafting articles, GitHub Copilot for code completion.
- **Summarization & extraction** — condensing legal, medical, or research documents.
- **Semantic search & RAG** — retrieval-augmented Q&A over private knowledge bases.

The key limitation of pure LLM applications is that they are **reactive**: they cannot independently decide to look something up, run code, remember a past interaction, or take multi-step action. This limitation motivated the shift toward **AI agents**.

2 AI Agents

An AI agent wraps an LLM in a **perceive** → **plan/reason** → **act** → **observe** loop, giving it autonomy to pursue a goal across multiple steps rather than producing one static response. The LLM acts as the reasoning core, deciding at each step whether to respond directly, call a tool, or revise its plan based on new observations. This transforms the model from a text generator into a goal-directed decision-maker.

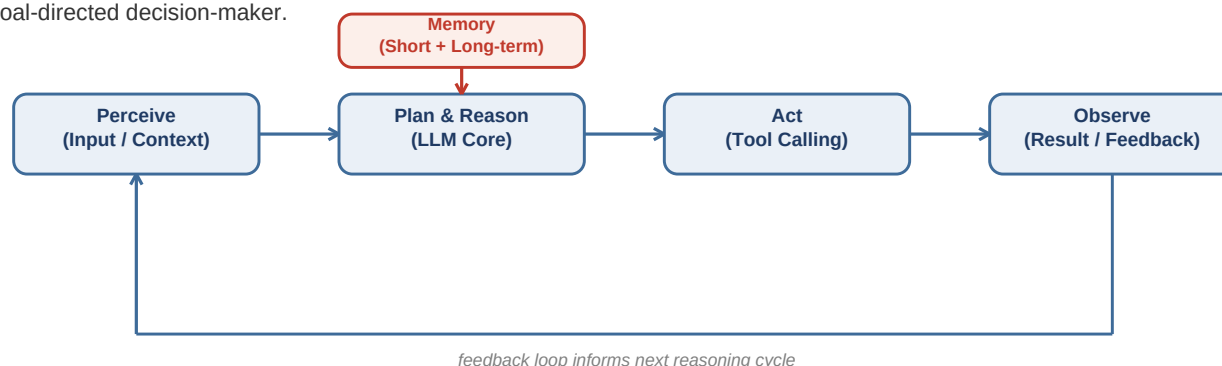


Figure 1. The core AI agent control loop, augmented by a memory subsystem.

Real-world example: Anthropic's computer-use agents and OpenAI's Operator perceive a screen, reason about the next UI action, act by clicking or typing, and observe the resulting screen state — repeating the loop until the task is complete.

3 Tool Calling

Tool calling (function calling) lets an LLM invoke external functions — APIs, databases, calculators, code interpreters, or web search — to extend its abilities beyond text prediction. The developer defines tools with a structured schema (name, description, parameters); the model outputs a structured request specifying which tool to call and with what arguments, and the system executes it and returns the result as new context.

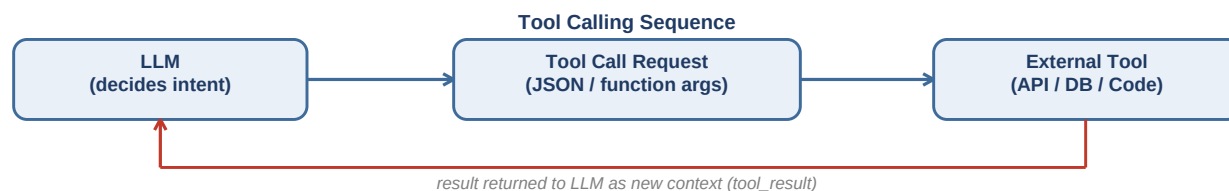


Figure 2. A single tool-calling exchange between an LLM and an external tool.

Real-world example: Claude and GPT models use tool calling to search the web, run Python code, query Google Drive, or place API calls to services like Slack and Salesforce, closing the gap between reasoning and real-world action.

4 Memory

Agents need memory to maintain coherence across steps and sessions. **Short-term (working) memory** lives in the model's context window — the running conversation and recent tool results. **Long-term memory** persists beyond a single session, typically using vector databases for semantic retrieval (RAG), structured logs of past actions (episodic memory), or summarized user preferences. Effective agents selectively write to and retrieve from long-term memory rather than replaying entire histories, which keeps context efficient.

Real-world example: Claude's memory feature and ChatGPT's memory both persist facts across conversations; frameworks like LangChain and LlamaIndex pair LLMs with vector stores (Pinecone, Chroma) for retrieval-augmented long-term memory.

5 Planning

Planning is how an agent decomposes a high-level goal into an ordered sequence of executable steps. Techniques include **Chain-of-Thought** prompting (reasoning step-by-step before answering), the **ReAct** framework (interleaving reasoning traces with actions), and **reflection** loops where the agent critiques and revises its own output. More advanced agents maintain an explicit task list or sub-goal tree, re-planning when a step fails or new information emerges.

Real-world example: AutoGPT and BabyAGI popularized autonomous planning loops that break a goal into sub-tasks, execute them with tools, and re-plan based on results; Devin (Cognition Labs) applies similar planning to autonomous software engineering.

6 Multi-Agent Systems

Complex tasks often exceed what a single agent can reliably handle. Multi-agent systems assign specialized roles — e.g., a researcher, a coder, and a reviewer — to separate agents coordinated by an orchestrator or manager agent. Agents communicate through shared messages, task queues, or a blackboard, allowing parallelism, specialization, and built-in critique (one agent checking another's work).

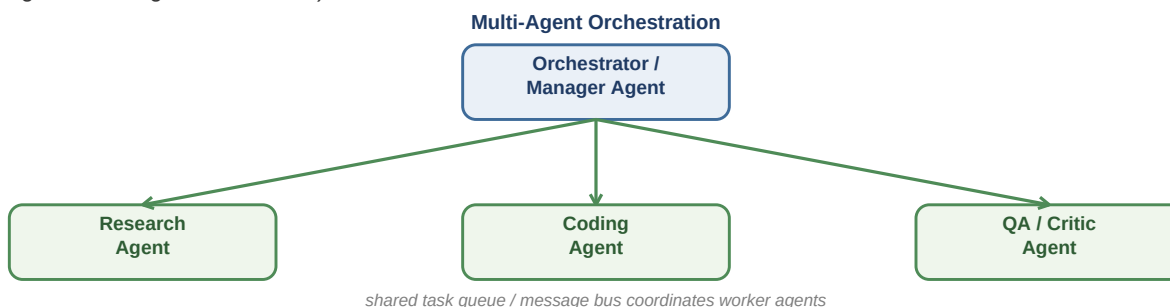


Figure 3. An orchestrator delegating sub-tasks to specialized worker agents.

Real-world example: Microsoft's AutoGen and CrewAI let developers define teams of role-based agents; Anthropic's Claude Agent SDK and multi-agent research systems similarly delegate sub-tasks to parallel sub-agents to speed up complex research.

7 Future of AI Engineering

- **Standardized interoperability** — protocols like Anthropic's Model Context Protocol (MCP) are standardizing how agents connect to tools and data sources, reducing custom integration work.
- **Agent evaluation & observability** — growing focus on tracing, benchmarking, and monitoring agent reliability in production, not just raw model accuracy.
- **Longer-horizon autonomy** — agents handling multi-hour or multi-day tasks with checkpointing and human-in-the-loop review.
- **Safety & alignment engineering** — sandboxing, permissioning, and guardrails become core engineering concerns as agents gain real-world capabilities.
- **Specialization** — a shift from one giant general agent toward composable, specialized agents orchestrated for domain-specific workflows (coding, research, customer support).

Overall, AI engineering is shifting from prompt-writing toward **systems engineering**: designing reliable architectures around LLMs — memory stores, tool integrations, evaluation harnesses, and multi-agent orchestration — rather than optimizing a single prompt.

References

- Yao, S., Zhao, J., Yu, D., et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629.
- Schick, T., Dwivedi-Yu, J., Dessì, R., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761.
- Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903.
- Park, J. S., O'Brien, J., Cai, C. J., et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. arXiv:2304.03442.
- Wu, Q., Bansal, G., Zhang, J., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. Microsoft Research.
- Anthropic (2024). Introducing the Model Context Protocol. anthropic.com/news/model-context-protocol.
- Anthropic (2025). Building Effective AI Agents. anthropic.com/research.
- Significant Gravitas (2023). AutoGPT: An Autonomous GPT-4 Experiment. GitHub repository.
- LangChain / LlamaIndex Documentation (2024). Agent, Tool, and Memory abstractions. docs.langchain.com, docs.llamaindex.ai.
- CrewAI Documentation (2024). Role-Based Multi-Agent Orchestration Framework. docs.crewai.com.